Universidad Mariano Gálvez de Guatemala Campus Central
Facultad de Ingeniería, Matemáticas y Ciencias Fiscas
Programación para la ciencia y la Ingeniería II

PROYECTO PPCYL2 AcadNet

Luis Fernando Lima De Leon
Lester Joel Argueta Patzan
Jose Javier Juarez Rivera
Jose Daniel Pedroza Juarez
Ingeniería Eléctrica; Jornada Vespertina
Guatemala 13 de abril de 2026

1. INTRODUCCIÓN

El proyecto **PPCYL2-AcadNet** representa una solución integral para la gestión y seguimiento del aprendizaje académico mediante una arquitectura web distribuida. El sistema facilita la conexión entre estudiantes y tutores, permitiendo una administración eficiente de calificaciones y horarios.

Este informe detalla la construcción de la plataforma utilizando una arquitectura de microservicios comunicados por el protocolo HTTP. Se destaca el uso de **Python** como lenguaje de programación, implementando **Flask** para el backend y **Django** para el frontend. El éxito de la solución radica en la integración de estructuras de datos avanzadas, específicamente una **Matriz Dispersa** desarrollada bajo el paradigma de Programación Orientada a Objetos (POO), y el uso de **Expresiones Regulares (Regex)** con **Pilas** para la extracción de información. Se confirma el cumplimiento de los hitos de entrega, incluyendo la documentación técnica y el versionamiento mediante releases en GitHub.

2. ARQUITECTURA DEL SISTEMA Y TECNOLOGÍAS

2.1 Desacoplamiento de Componentes

La arquitectura de PPCYL2-AcadNet se basa en la separación de responsabilidades:

- **Servidor de Aplicaciones (Backend - Flask):** Actúa como el motor lógico. Es el encargado de procesar archivos XML, gestionar la memoria RAM mediante estructuras de datos personalizadas y exponer servicios REST.
- **Interfaz de Usuario (Frontend - Django):** Proporciona una experiencia de usuario robusta y segura. Utiliza el motor de plantillas de Django para renderizar dashboards personalizados según el rol del usuario (Administrador, Tutor o Estudiante).

2.2 Flujo de Comunicación HTTP

La comunicación entre ambos módulos se realiza mediante peticiones asíncronas. El backend funciona de manera apátrida (stateless), recibiendo parámetros y archivos para devolver respuestas en formato JSON para la interfaz de usuario y XML para reportes de sistema.

3. LÓGICA DE PROCESAMIENTO Y EXPRESIONES REGULARES

3.1 Análisis de Horarios con Estructuras de Pila

Uno de los requisitos críticos fue el procesamiento de horarios de tutores. Debido a que la información puede presentarse de forma desestructurada, se implementó un motor de análisis basado en:

1. **Patrones Regex:** Se diseñaron expresiones regulares para identificar días de la semana y rangos horarios (HH:MM - HH:MM).
2. **Validación con Pilas:** Se utilizó una estructura de datos de **Pila (Stack)** para gestionar el anidamiento de etiquetas XML y para validar que los horarios de los tutores no se solaparan durante el procesamiento secuencial.

3.2 Transformación de Datos XML

El sistema cuenta con un módulo de "Carga Masiva" que analiza archivos XML de entrada. Este módulo valida la integridad referencial (por ejemplo, que no se asigne un estudiante a un curso que no existe) antes de persistir la información en las estructuras de datos.

4. MATRIZ DISPERSA

4.1 Justificación de la Estructura POO

En un entorno académico, las matrices de calificaciones suelen tener muchos espacios vacíos (no todos los alumnos cursan todas las materias). Una matriz tradicional desperdiciaría memoria. Por ello, se implementó una **Matriz Dispersa**.

4.2 Componentes de la Matriz

Se definieron tres clases fundamentales:

- **Clase Nodo:** Contiene la información de la nota y punteros (Norte, Sur, Este, Oeste).
- **Clase Encabezado:** Una lista doblemente enlazada que organiza los índices de filas (actividades) y columnas (estudiantes).
- **Clase Matriz:** Orquestadora de la lógica de inserción ortogonal, garantizando que cada nodo se ubique correctamente en su intersección.

5. ESTRUCTURA

Estructura del proyecto

```text
PPCYL2_PSemestre_Proyecto2_[1492]/
│
├── readme.md
├── .gitignore
│
├── backend/
│   ├── app.py
│   ├── requirements.txt
│   ├── rutas/
│   │   └── __init__.py
│   ├── estructuras/
│   │   ├── __init__.py
│   │   ├── nodo.py
│   │   └── matriz_dispersa.py
│   ├── analizador/
│   │   ├── __init__.py
│   │   ├── lector_xml.py
│   │   ├── lector_configuracion.py
│   │   ├── lector_horarios.py
│   │   ├── reporte_graphviz.py
│   │   └── generador_salida_configuracion.py
│   └── archivos_datos/
│       ├── entrada/
│       │   ├── configuracion.xml
│       │   ├── horarios.xml
│       │   └── notas.xml
│       └── salida/
│           ├── configuraciones_aplicadas.xml
│           └── matriz_dispersa.png
│
└── frontend/
    ├── manage.py
    ├── requirements.txt
    ├── db.sqlite3
    ├── acadnet_project/
    │   ├── __init__.py
    │   ├── settings.py
    │   ├── urls.py
    │   ├── asgi.py
    │   └── wsgi.py
    └── app_cliente/
        ├── __init__.py
        ├── admin.py
        ├── apps.py
        ├── decorators.py
        ├── forms.py
        ├── models.py
        ├── signals.py
        ├── urls.py
        ├── views.py
        ├── migrations/
        ├── templates/
        │   ├── login.html
        │   ├── admin_dashboard.html
        │   ├── tutor_dashboard.html
        │   └── estudiante_dashboard.html
        └── static/
            └── css/
```

6. APÉNDICE I: MODELOS Y DISEÑOS

5.1 Diagrama de Clases UML

El diseño de la solución se rige por un diagrama de clases que detalla la herencia y composición. Las clases de usuario (Tutor, Estudiante) heredan de una clase base, mientras que la estructura de la matriz se mantiene aislada para permitir su reutilización en otros módulos.

5.2 Modelo de Salida XML

El archivo de salida generado tras la carga masiva sigue el estándar definido en el enunciado:

XML

```
<respuesta>
    <estudiantes_cargados>X</estudiantes_cargados>
    <tutores_cargados>Y</tutores_cargados>
    <errores>
        <error>Descripción del error de curso</error>
    </errores>
</respuesta>
```

7. ANEXO II Y CONCLUSIONES

6.1 Reportes y Visualización de Datos

El sistema consume los datos de la Matriz Dispersa para generar reportes estadísticos. Se implementaron gráficos de barras para mostrar el promedio de notas por actividad evaluativa, permitiendo a los administradores identificar áreas de mejora académica.

6.2 Conclusiones Finales

- **Escalabilidad:** El uso de Flask/Django permite que la plataforma crezca sin comprometer el rendimiento.
- **Integridad:** Las expresiones regulares aseguraron una carga de datos libre de errores sintácticos.
- **Metodología:** Los releases semanales en el repositorio `PPCYL2_PSemestre_Proyecto2_1492` garantizan la trazabilidad del desarrollo y la transparencia en la autoría del código.